Prolog Learning Guide for Smart Device Repair Diagnosis

Project: Smart Laptop & Phone Repair Diagnosis Expert System
Main learning goal: Understand Prolog through a practical rule-based expert system
Backend: SWI-Prolog
Frontend: Python Tkinter, only used to collect input and display output

---

1. Project Purpose

This assignment is designed to show how Prolog can be used in a real application. The project is an expert system for diagnosing common laptop and phone repair issues.

The user selects a device and symptoms. The Prolog knowledge base compares those symptoms with known repair rules and returns possible faults, match score, severity, cost level, backup warning, and repair advice.

The most important point:

> The Python program is only the interface. The actual intelligence and reasoning are done in Prolog.

---

2. What Is an Expert System?

An expert system is a program that imitates decision-making by using expert knowledge.

In this project, the "expert knowledge" is repair knowledge:

- If a laptop has overheating and loud fan, it may have a cooling problem.
- If a phone has water damage, no power, and no charging, it may have water damage.
- If storage-related symptoms appear, the user should backup data.

This is suitable for Prolog because Prolog is good at:

- Representing facts
- Writing rules
- Searching possible answers
- Matching patterns
- Using backtracking
- Working with lists

---

3. System Architecture

The system has two main parts:

```
User
 |
 v
Python Tkinter GUI
 |
 v
SWI-Prolog backend
 |
 v
Diagnosis result
```

Python sends a query like this to Prolog:

```
run_diagnosis(laptop, [overheating,loud_fan,random_shutdown]).
```

Prolog then:

1. Clears old symptoms.
2. Adds the selected symptoms as temporary facts.
3. Matches symptoms with fault rules.
4. Calculates score.
5. Sorts possible faults.
6. Prints the results.

---

4. Main Prolog Files

File	Purpose
`repair_expert.pl`	Main file that loads all Prolog modules
`repair_devices_symptoms.pl`	Stores supported devices and symptoms
`repair_faults.pl`	Stores fault details such as severity, cost, advice, backup
`repair_rules.pl`	Stores rules that connect faults with symptom lists
`repair_engine.pl`	Main reasoning engine: matching, scoring, sorting
`repair_decision.pl`	Gives repair decision based on cost, severity, and backup
`repair_io.pl`	Formats output for terminal and Python
`repair_admin.pl`	Adds and removes custom repair cases dynamically

---

5. Prolog Basics You Must Know

5.1 Facts

A fact is something that is declared as true.

Example:

```
device(laptop).
device(phone).
```

This means:

- `laptop` is a supported device.
- `phone` is a supported device.

Another example:

```
symptom(laptop, overheating, "Overheating").
```

This means:

> A laptop can have a symptom called `overheating`, and its display label is `"Overheating"`.

5.2 Predicates

A predicate is a relation.

Example:

```
device(laptop).
```

Here:

- `device` is the predicate name.
- `laptop` is the argument.
- `device/1` means the predicate has 1 argument.

Example:

```
symptom(laptop, overheating, "Overheating").
```

Here:

- `symptom` is the predicate name.

- It has 3 arguments.
- So it is called `symptom/3`.

5.3 Atoms

Atoms are constant names in Prolog.

Examples:

```
laptop
phone
overheating
battery_failure
critical
yes
no
```

Atoms usually start with lowercase letters.

5.4 Variables

Variables start with uppercase letters.

Examples:

```
Device
Fault
Symptoms
Score
Result
```

Example query:

```
device(D).
```

Prolog can answer:

```
D = laptop ;
D = phone.
```

This means Prolog searches all possible values for `D`.

5.5 Rules

A rule has a head and a body.

General form:

```
head :-
    condition1,
    condition2.
```

Read `:-` as "if".

Example:

```
likely_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    score_fault(Device, Fault, Score, MatchCount, Total, Matched),
    Score >= 50.
```

Meaning:

> A fault is likely if it has a score and the score is at least 50.

5.6 Queries

A query asks Prolog to prove something.

Example:

```
?- device(laptop).
```

Answer:

```
true.
```

Example:

```
?- device(tablet).
```

Answer:

```
false.
```

Because `tablet` is not listed as a supported device.

---

6. Main Knowledge Base

The project knowledge base has three main types of knowledge:

1. Devices and symptoms
2. Fault information
3. Diagnostic rules

---

7. Devices and Symptoms

File:

```
repair_devices_symptoms.pl
```

Example:

```
device(laptop).
device(phone).
```

This declares the supported devices.

Symptoms are represented using `symptom/3`.

```
symptom(laptop, no_power, "Does not power on").
symptom(laptop, overheating, "Overheating").
symptom(phone, cracked_screen, "Cracked screen").
symptom(phone, water_damage, "Water damage").
```

Format:

```
symptom(Device, SymptomAtom, HumanReadableLabel).
```

Example explanation:

```
symptom(phone, water_damage, "Water damage").
```

This means:

- Device: `phone`
- Internal symptom atom: `water_damage`
- Display label: `"Water damage"`

The internal atom is used by Prolog rules. The human-readable label is shown in the interface.

---

8. Fault Information

File:

```
repair_faults.pl
```

Fault information is stored using `fault_info/7`.

Format:

```
fault_info(FaultAtom, Device, Label, Severity, Cost, Advice, BackupNeeded).
```

Example:

```
fault_info(storage_failure, laptop, "Hard Disk / SSD Failure", critical, high,
    "Stop using the device heavily. Backup data immediately and replace storage.", yes).
```

Explanation:

Argument	Value	Meaning
`FaultAtom`	`storage_failure`	Internal fault name
`Device`	`laptop`	Applies to laptop
`Label`	`"Hard Disk / SSD Failure"`	Display name
`Severity`	`critical`	Very serious fault
`Cost`	`high`	Expensive repair
`Advice`	text	Repair recommendation
`BackupNeeded`	`yes`	User should backup data

This is not a diagnosis rule yet. It only stores information about the fault.

---

9. Diagnostic Rules

File:

```
repair_rules.pl
```

Diagnostic rules use `fault_symptoms/3`.

Format:

```
fault_symptoms(Device, FaultAtom, RequiredSymptoms).
```

Example:

```
fault_symptoms(laptop, cooling_problem,
    [overheating, loud_fan, random_shutdown, slow_performance]).
```

Meaning:

> For a laptop, the symptoms overheating, loud fan, random shutdown, and slow performance may indicate a cooling problem.

Another example:

```
fault_symptoms(phone, phone_water_damage,
    [water_damage, no_power, no_charging]).
```

Meaning:

> For a phone, water damage, no power, and no charging may indicate phone water damage.

This is the core repair knowledge of the system.

10. Difference Between `fault_info/7` and `fault_symptoms/3`

This is important for explaining the design.

`fault_info/7` describes the fault.

Example:

```
fault_info(cooling_problem, laptop, "Cooling System Problem", high, medium,
    "Clean the fan, replace thermal paste, and check air vents.", no).
```

`fault_symptoms/3` explains how to detect the fault.

Example:

```
fault_symptoms(laptop, cooling_problem,
    [overheating, loud_fan, random_shutdown, slow_performance]).
```

Together they mean:

> If the selected symptoms match the rule for `cooling_problem`, the system can also retrieve its label, severity, cost, advice, and backup warning.

---

11. Dynamic Predicates

File:

```
repair_expert.pl
```

Important declarations:

```
:- dynamic observed_symptom/1.
:- dynamic fault_info/7.
:- dynamic fault_symptoms/3.
```

`dynamic` means the predicate can be changed while the program is running.

In this project:

- `observed_symptom/1` changes every time the user selects symptoms.
- `fault_info/7` can be extended with custom cases.
- `fault_symptoms/3` can be extended with custom diagnostic rules.

Example:

```
assertz(observed_symptom(overheating)).
```

This adds:

```
observed_symptom(overheating).
```

to Prolog's working memory.

Example:

```
retractall(observed_symptom(_)).
```

This removes all selected symptoms.

Presentation line:

> I used dynamic predicates to store temporary user symptoms and to support custom repair cases.

---

12. Multifile Predicates

File:

```
repair_expert.pl
```

Declarations:

```
:- multifile fault_info/7.
:- multifile fault_symptoms/3.
```

`multifile` means the same predicate can be defined in multiple files.

Why it is used:

- Normal faults are stored in `repair_faults.pl`.
- Diagnostic rules are stored in `repair_rules.pl`.
- Custom cases can be stored in `custom_cases.pl`.

This keeps the project modular and easier to maintain.

Presentation line:

> I used multifile predicates so that the same knowledge predicate can be extended from separate files, including custom user cases.

---

13. Loading Modules

File:

```
repair_expert.pl
```

The main file loads other files using `ensure_loaded/1`.

```
:- ensure_loaded('repair_devices_symptoms.pl').
:- ensure_loaded('repair_faults.pl').
:- ensure_loaded('repair_rules.pl').
:- ensure_loaded('repair_decision.pl').
:- ensure_loaded('repair_engine.pl').
:- ensure_loaded('repair_io.pl').
:- ensure_loaded('repair_admin.pl').
```

This means only `repair_expert.pl` needs to be loaded manually.

Example:

```
?- [repair_expert].
```

After that, all other predicates are available.

---

14. Main Diagnosis Flow

The main entry point is:

```
run_diagnosis(Device, Symptoms).
```

Example:

```
run_diagnosis(laptop, [overheating,loud_fan,random_shutdown]).
```

Defined in `repair_io.pl`:

```
run_diagnosis(Device, Symptoms) :-
    clear_observations,
    add_symptom_list(Symptoms),
    diagnose(Device, Results),
    print_results(Results),
    halt.
```

Step-by-step:

1. `clear_observations`
    - Removes previous selected symptoms.
2. `add_symptom_list(Symptoms)`
    - Adds the new selected symptoms.
3. `diagnose(Device, Results)`
    - Runs the expert system logic.
4. `print_results(Results)`
    - Prints output in a format Python can read.
5. `halt`
    - Stops SWI-Prolog after completing the query.

---

15. Working Memory

Working memory means temporary facts used during reasoning.

In this project:

```
observed_symptom(overheating).
observed_symptom(loud_fan).
observed_symptom(random_shutdown).
```

These are not permanent repair rules. They are temporary facts representing what the user selected.

Before a new diagnosis:

```
clear_observations :- retractall(observed_symptom(_)).
```

This clears old facts.

Then:

```
add_observed_symptom(Symptom) :-
    \+ observed_symptom(Symptom),
    assertz(observed_symptom(Symptom)).
add_observed_symptom(_).
```

This adds symptoms only if they are not already present.

Important symbol:

```
\+
```

This means "not provable" or negation as failure.

So:

```
\+ observed_symptom(Symptom)
```

means:

> The symptom has not already been added.

---

16. Recursion in `add_symptom_list/1`

Code:

```
add_symptom_list([]).
add_symptom_list([H|T]) :-
    add_observed_symptom(H),
    add_symptom_list(T).
```

This is recursive list processing.

Base case:

```
add_symptom_list([]).
```

This means when the list is empty, stop.

Recursive case:

```
add_symptom_list([H|T]) :-
    add_observed_symptom(H),
    add_symptom_list(T).
```

`[H|T]` splits a list into:

- `H` : first item
- `T` : remaining list

Example:

```
[overheating,loud_fan,random_shutdown]
```

First call:

```
H = overheating
T = [loud_fan, random_shutdown]
```

Second call:

```
H = loud_fan
T = [random_shutdown]
```

Third call:

```
H = random_shutdown
T = []
```

Then the base case stops recursion.

Presentation line:

> I used recursion to process the selected symptom list and assert each symptom into working memory.

---

17. Matching Symptoms

Code from `repair_engine.pl` :

```
matched_symptoms(Required, Matched) :-
    findall(S, (member(S, Required), observed_symptom(S)), Matched).
```

This is one of the most important predicates.

It compares:

- `Required` : symptoms needed for a fault
- `observed_symptom(S)` : symptoms selected by the user
- `Matched` : symptoms that exist in both lists

Example:

Required symptoms:

```
[overheating, loud_fan, random_shutdown, slow_performance]
```

Observed symptoms:

```
observed_symptom(overheating).
observed_symptom(loud_fan).
observed_symptom(random_shutdown).
```

Matched result:

```
[overheating, loud_fan, random_shutdown]
```

`member/2`

```
member(S, Required)
```

checks whether `S` is inside the list `Required`.

`findall/3`

```
findall(S, Goal, Matched)
```

means:

> Find all values of `S` that satisfy `Goal`, and collect them into `Matched`.

In this project:

```
findall(S, (member(S, Required), observed_symptom(S)), Matched)
```

means:

> Collect every symptom `S` that is both required by the fault and observed by the user.

---

18. Scoring a Fault

Code:

```
score_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    fault_symptoms(Device, Fault, Required),
    matched_symptoms(Required, Matched),
    length(Matched, MatchCount),
    length(Required, Total),
    Total > 0,
    MatchCount > 0,
    Score is (MatchCount * 100) // Total.
```

Step-by-step:

1. Get required symptoms:

```
fault_symptoms(Device, Fault, Required)
```

2. Find matched symptoms:

```
matched_symptoms(Required, Matched)
```

3. Count matched symptoms:

```
length(Matched, MatchCount)
```

4. Count total required symptoms:

```
length(Required, Total)
```

5. Calculate percentage:

```
Score is (MatchCount * 100) // Total
```

Example:

```
Matched symptoms = 3
Total required symptoms = 4
Score = 3 * 100 // 4
Score = 75
```

`//` means integer division.

So `75.0` becomes `75`.

Presentation line:

> The score is calculated as matched symptoms divided by required symptoms, multiplied by 100.

---

19. Likely Fault Rule

Code:

```
likely_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    score_fault(Device, Fault, Score, MatchCount, Total, Matched),
    Score >= 50.
```

This means:

> A fault is considered likely only if at least 50% of its symptoms match.

Example:

If cooling problem has 4 symptoms and the user matches 3:

```
3/4 = 75%
```

It is likely.

If only 1 out of 4 matches:

```
1/4 = 25%
```

It is not shown.

---

20. Creating a Result Row

Code:

```
make_result(Device,
            result(Fault, Score, Label, Severity, Cost, BackupNeeded, Advice, Matched, MatchCount, Total)) :-
    likely_fault(Device, Fault, Score, MatchCount, Total, Matched),
    fault_info(Fault, Device, Label, Severity, Cost, Advice, BackupNeeded).
```

This connects two parts:

1. Diagnosis rule:

```
likely_fault(...)
```

2. Fault information:

```
fault_info(...)
```

If a fault is likely, Prolog retrieves its display label, severity, cost, advice, and backup warning.

The result is stored as a compound term:

```
result(Fault, Score, Label, Severity, Cost, BackupNeeded, Advice, Matched, MatchCount, Total)
```

Presentation line:

> `make_result/2` combines inference output with fault metadata to create a complete diagnosis result.

---

21. Backtracking

Backtracking is one of the most important Prolog concepts.

Prolog does not only check one rule. It tries one possible solution, then automatically goes back and tries other possible solutions.

In this project, Prolog checks every `fault_symptoms/3` rule for the selected device.

Example query:

```
fault_symptoms(laptop, Fault, Symptoms).
```

Possible answers:

```
Fault = battery_failure
Symptoms = [battery_drain, random_shutdown, no_power] ;

Fault = charger_or_port_issue
Symptoms = [no_charging, no_power] ;

Fault = cooling_problem
Symptoms = [overheating, loud_fan, random_shutdown, slow_performance] ;
...
```

This happens because Prolog backtracks through all matching facts.

Presentation line:

> Backtracking allows the system to search all possible faults instead of stopping at the first match.

---

22. Collecting All Diagnoses

Code:

```
diagnose(Device, ResultsSorted) :-
    findall(Result, make_result(Device, Result), Results),
    list_to_set(Results, Unique),
    predsort(compare_score, Unique, ResultsSorted).
```

This does three things:

Step 1: Collect all possible results

```
findall(Result, make_result(Device, Result), Results)
```

This uses backtracking internally and collects every result.

Step 2: Remove duplicates

```
list_to_set(Results, Unique)
```

This removes repeated result terms.

Step 3: Sort by score

```
predsort(compare_score, Unique, ResultsSorted)
```

This sorts the diagnosis results using a custom comparison predicate.

Presentation line:

> `diagnose/2` is the main reasoning predicate. It collects all likely faults, removes duplicates, and sorts them by confidence score.

---

23. Sorting Results

Code:

```
compare_score(Order, result(_, ScoreA, _, _, _, _, _, _, _, _),
                     result(_, ScoreB, _, _, _, _, _, _, _, _)) :-
    ( ScoreA > ScoreB -> Order = '<'
    ; ScoreA < ScoreB -> Order = '>'
    ; Order = '<'
    ).
```

This compares two result rows.

If `ScoreA` is greater than `ScoreB`, then `ScoreA` should come first.

The symbols may look reversed because `predsort/3` expects:

- `<` means first item comes before second item
- `>` means first item comes after second item

So a higher score is placed earlier.

---

24. Repair Decision Logic

File:

```
repair_decision.pl
```

Rules:

```
repair_decision(high, critical, consider_replacing).
repair_decision(high, high, replace_faulty_part).
repair_decision(medium, critical, backup_and_repair).
repair_decision(low, low, repair_device).
```

Format:

```
repair_decision(Cost, Severity, Decision).
```

Example:

```
repair_decision(high, critical, consider_replacing).
```

Meaning:

> If cost is high and severity is critical, compare repair vs replacement.

Backup Priority

Code:

```
get_decision(Cost, Severity, BackupNeeded, Decision) :-
    repair_decision(Cost, Severity, BaseDecision),
    ( BackupNeeded = yes
      -> Decision = backup_and_repair
      ;  Decision = BaseDecision
    ).
```

This means:

- First get the base decision using cost and severity.
- If backup is needed, override the decision to `backup_and_repair`.
- Otherwise, use the base decision.

Important operator:

```
Condition -> Then ; Else
```

This is Prolog's if-then-else structure.

Presentation line:

> The repair decision is rule-based. It uses cost and severity, but data backup has priority when `BackupNeeded` is yes.

---

25. Output Formatting

File:

```
repair_io.pl
```

Code:

```
print_results([]) :-
    format('NO_RESULT|No strong fault matched|Try selecting more symptoms|~n').
```

This handles the case where no likely fault is found.

Code:

```
print_results(Results) :-
    forall(member(R, Results), print_result_line(R)).
```

This prints every result.

`forall/2`

```
forall(Condition, Action)
```

means:

> For every solution of `Condition`, perform `Action`.

In this project:

```
forall(member(R, Results), print_result_line(R))
```

means:

> For every result `R` in the results list, print one result line.

Result Format

Output uses `|` separators:

```
RESULT|fault|score|match_count|total|severity|cost|backup|label|advice|matched|decision
```

This makes it easy for Python to split and display the result.

---

26. Dynamic Custom Cases

File:

```
repair_admin.pl
```

Code:

```
add_custom_case(Device, Fault, Label, Symptoms, Severity, Cost, Advice, BackupNeeded) :-
    assertz(fault_info(Fault, Device, Label, Severity, Cost, Advice, BackupNeeded)),
    assertz(fault_symptoms(Device, Fault, Symptoms)).
```

This dynamically adds:

1. Fault information
2. Fault symptom rule

Example:

```
add_custom_case(
    laptop,
    gpu_failure,
    "GPU Failure",
    [black_screen, overheating, display_flicker],
    high,
    high,
    "Check GPU and motherboard. Service center diagnosis recommended.",
    no
).
```

Remove case:

```
remove_case(Fault) :-
    retractall(fault_info(Fault, _, _, _, _, _, _)),
    retractall(fault_symptoms(_, Fault, _)).
```

The underscore `_` means anonymous variable. It means:

> I do not care what value is here.

Presentation line:

> Custom cases show dynamic knowledge base modification using `assertz/1` and `retractall/1`.

---

27. Important Prolog Built-ins Used

Built-in	Used for	Example
`assertz/1`	Add a fact dynamically	`assertz(observed_symptom(overheating))`
`retractall/1`	Remove matching facts	`retractall(observed_symptom(_))`
`findall/3`	Collect all solutions	`findall(S, Goal, List)`
`member/2`	Check item in list	`member(S, Required)`
`length/2`	Count list items	`length(Matched, Count)`
`list_to_set/2`	Remove duplicates	`list_to_set(Results, Unique)`
`predsort/3`	Sort using custom rule	`predsort(compare_score, Unique, Sorted)`
`forall/2`	Perform action for all items	`forall(member(R, Results), print_result_line(R))`

Built-in	Used for	Example
`atomic_list_concat/3`	Join list items into text	`atomic_list_concat(Matched, ',', Text)`
`format/2`	Print formatted output	`format('Result: ~w', [Fault])`
`exists_file/1`	Check if file exists	`exists_file('custom_cases.pl')`
`consult/1`	Load Prolog file	`consult('custom_cases.pl')`
`halt/0`	Stop Prolog	`halt`

---

28. Important Operators and Symbols

Symbol	Meaning	Example
`:-`	if / rule definition	`a :- b.`
`,`	AND	`device(D), symptom(D,S,L)`
`;`	OR / else part	`Then ; Else`
`.`	End of fact or rule	`device(laptop).`
`_`	Anonymous variable	`observed_symptom(_)`
`\+`	Negation as failure	`\+ observed_symptom(S)`
`=`	Unification	`BackupNeeded = yes`
`is`	Arithmetic evaluation	`Score is X + Y`
`>=`	Greater than or equal	`Score >= 50`
`[H	T]`	Head and tail of list
`[]`	Empty list	`add_symptom_list([])`

---

29. Unification

Unification is Prolog's pattern matching process.

Example:

```
device(D).
```

Prolog tries to match this query with facts:

```
device(laptop).
device(phone).
```

So:

```
D = laptop
D = phone
```

Another example:

```
fault_info(Fault, laptop, Label, Severity, Cost, Advice, Backup).
```

This asks:

> Find all fault information where the device is laptop.

Prolog fills the variables with matching values.

Presentation line:

> Prolog uses unification to match query patterns with facts and rules in the knowledge base.

---

30. Negation as Failure

Code:

```
\+ observed_symptom(Symptom)
```

This means:

> Prolog cannot prove that this symptom already exists.

It is used to prevent duplicate observed symptoms:

```
add_observed_symptom(Symptom) :-
    \+ observed_symptom(Symptom),
    assertz(observed_symptom(Symptom)).
add_observed_symptom(_).
```

If the symptom is already present, the first rule fails. Then the second rule succeeds:

```
add_observed_symptom(_).
```

This prevents an error and allows the program to continue.

---

31. Example Diagnosis Walkthrough

Query:

```
run_diagnosis(laptop, [overheating,loud_fan,random_shutdown]).
```

Step 1: Clear old symptoms

```
clear_observations
```

removes all previous:

```
observed_symptom(_)
```

Step 2: Add new symptoms

```
add_symptom_list([overheating,loud_fan,random_shutdown])
```

creates:

```
observed_symptom(overheating).
observed_symptom(loud_fan).
observed_symptom(random_shutdown).
```

Step 3: Check cooling problem rule

Rule:

```
fault_symptoms(laptop, cooling_problem,
    [overheating, loud_fan, random_shutdown, slow_performance]).
```

Matched:

```
[overheating, loud_fan, random_shutdown]
```

Score:

```
3 / 4 * 100 = 75
```

Since 75 is greater than 50, it is a likely fault.

Step 4: Get fault information

```
fault_info(cooling_problem, laptop, "Cooling System Problem", high, medium,
    "Clean the fan, replace thermal paste, and check air vents.", no).
```

Step 5: Get repair decision

Cost:

```
medium
```

Severity:

```
high
```

Backup:

```
no
```

Decision:

```
repair_device
```

Display label:

```
Repair Device
```

---

32. Sample Queries to Practice

Load the project:

```
?- [repair_expert].
```

List devices:

```
?- device(D).
```

List all laptop symptoms:

```
?- symptom(laptop, S, Label).
```

List all phone symptoms:

```
?- symptom(phone, S, Label).
```

Find symptoms for a fault:

```
?- fault_symptoms(laptop, cooling_problem, Symptoms).
```

Find fault information:

```
?- fault_info(storage_failure, laptop, Label, Severity, Cost, Advice, Backup).
```

Manual diagnosis:

```
?- clear_observations,
   add_symptom_list([overheating,loud_fan,random_shutdown]),
   diagnose(laptop, Results).
```

Run full diagnosis:

```
?- run_diagnosis(laptop, [overheating,loud_fan,random_shutdown]).
```

Phone water damage:

```
?- run_diagnosis(phone, [water_damage,no_power,no_charging]).
```

Laptop storage failure:

```
?- run_diagnosis(laptop, [boot_failure,clicking_sound,slow_performance]).
```

Phone display damage:

```
?- run_diagnosis(phone, [cracked_screen,black_screen,touch_not_working]).
```

---

33. Expected Demo Scenarios

Scenario 1: Laptop Cooling Problem

Input:

```
[overheating,loud_fan,random_shutdown]
```

Expected:

```
Cooling System Problem
Score: 75
Severity: high
Cost: medium
Decision: Repair Device
```

Scenario 2: Laptop Storage Failure

Input:

```
[boot_failure,clicking_sound,slow_performance]
```

Expected:

```
Hard Disk / SSD Failure
Score: 100
Severity: critical
Cost: high
Backup: yes
Decision: Backup Data & Repair
```

Scenario 3: Phone Water Damage

Input:

```
[water_damage,no_power,no_charging]
```

Expected:

```
Water Damage
Score: 100
Severity: critical
Cost: high
Backup: yes
Decision: Backup Data & Repair
```

Scenario 4: Phone Display Damage

Input:

```
[cracked_screen,black_screen,touch_not_working]
```

Expected:

```
Display or Touch Panel Damage
Score: 100
Severity: high
Cost: high
Decision: Replace Faulty Part
```

---

34. How to Explain the Whole Prolog System

Use this explanation in your presentation:

> My project is a rule-based expert system built using Prolog. It diagnoses laptop and phone faults from selected symptoms. I represented devices, symptoms, fault information, and diagnostic knowledge as Prolog facts. The main diagnostic rules use `fault_symptoms/3`, where each fault is connected to a list of symptoms. When the user selects symptoms, the system stores them dynamically as `observed_symptom/1` facts using `assertz/1`. The diagnosis engine compares observed symptoms with each fault's required symptoms using `member/2` and `findall/3`. It calculates a score based on how many symptoms match, filters faults with at least 50% match, and sorts them using `predsort/3`. Finally, another rule base uses cost, severity, and backup risk to recommend whether to repair, replace a part, or backup data first.

---

35. Viva Questions and Answers

Q1. Why did you use Prolog for this project?

Because the problem is rule-based. Device repair diagnosis can be represented using facts and rules, such as symptoms indicating faults. Prolog is suitable because it supports logical inference, backtracking, pattern matching, and list processing.

Q2. What are the facts in your system?

Examples of facts are:

```
device(laptop).
symptom(laptop, overheating, "Overheating").
fault_info(cooling_problem, laptop, "Cooling System Problem", high, medium, Advice, no).
fault_symptoms(laptop, cooling_problem, [overheating,loud_fan,random_shutdown,slow_performance]).
```

These facts store devices, symptoms, fault details, and diagnostic knowledge.

Q3. What is the main rule in your system?

The main reasoning rule is `likely_fault/6`:

```
likely_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    score_fault(Device, Fault, Score, MatchCount, Total, Matched),
    Score >= 50.
```

It says a fault is likely if its symptom match score is at least 50%.

Q4. What is dynamic knowledge base?

A dynamic knowledge base allows facts to be added or removed while the program is running. In my project, selected symptoms are added using `assertz/1` and cleared using `retractall/1`.

Q5. Where do you use `assertz/1`?

I use it in:

```
add_observed_symptom(Symptom) :-
    \+ observed_symptom(Symptom),
    assertz(observed_symptom(Symptom)).
```

It adds the user's selected symptom as a temporary fact.

Q6. Where do you use `retractall/1`?

I use it in:

```
clear_observations :- retractall(observed_symptom(_)).
```

This removes all previous symptoms before a new diagnosis.

Q7. How does your system calculate confidence?

It counts how many symptoms matched and divides by the total required symptoms:

```
Score is (MatchCount * 100) // Total.
```

Example:

```
3 matched symptoms out of 4 total symptoms = 75%
```

Q8. What is backtracking?

Backtracking is Prolog's ability to search for alternative solutions. In this project, Prolog checks all possible fault rules for a device and can return multiple possible diagnoses.

Q9. Where do you use `findall/3`?

I use it in two places:

```
matched_symptoms(Required, Matched) :-
    findall(S, (member(S, Required), observed_symptom(S)), Matched).
```

This collects matched symptoms.

```
diagnose(Device, ResultsSorted) :-
    findall(Result, make_result(Device, Result), Results),
    ...
```

This collects all diagnosis results.

Q10. What does `member/2` do?

`member/2` checks whether an item is inside a list.

Example:

```
member(overheating, [overheating,loud_fan]).
```

This is true.

In my project, it checks whether a required symptom is in the fault's symptom list.

Q11. Why do you use lists?

I use lists to store symptoms for each fault.

Example:

```
[overheating, loud_fan, random_shutdown, slow_performance]
```

Lists make it easy to count symptoms, compare symptoms, and calculate scores.

Q12. What is the use of `fault_info/7`?

It stores detailed information about a fault, including label, severity, cost, advice, and backup warning.

Q13. What is the use of `fault_symptoms/3`?

It defines which symptoms are connected to each fault. It is the main diagnostic rule base.

Q14. What does `BackupNeeded` do?

It tells the system whether the user should backup data before repair. If `BackupNeeded` is `yes`, the final decision becomes `Backup Data & Repair`.

Q15. What is the role of Python?

Python is only the user interface. It collects selected symptoms and sends them to Prolog. The diagnosis logic is done in Prolog.

Q16. What is `multifile`?

`multifile` allows one predicate to be defined across multiple files. I use it for `fault_info/7` and `fault_symptoms/3`, so custom cases can be stored separately.

Q17. What is `ensure_loaded/1`?

It loads another Prolog file. The main file `repair_expert.pl` uses it to load all modules.

Q18. What is the limitation of your project?

It is rule-based and depends on predefined symptoms and faults. It does not use real sensor data or machine learning. However, it is easy to explain, extend, and suitable for learning logic programming.

Q19. How can the system be improved?

Possible improvements:

- Add more device types such as tablets and printers.
- Add more symptoms and repair rules.
- Store custom cases in a database.
- Add probability or weighted symptoms.
- Add a web interface.
- Add technician login and repair history.

Q20. What Prolog concepts does your project demonstrate?

It demonstrates:

- Facts
- Rules
- Queries
- Unification
- Backtracking
- Lists
- Recursion
- Dynamic predicates
- `assertz/1`
- `retractall/1`
- `findall/3`
- `member/2`
- `forall/2`
- Sorting with `predsort/3`
- Modular knowledge base design

---

36. Strong Final Summary

Memorize this:

> This project shows how Prolog can be used to build a practical expert system. The system stores repair knowledge as facts and rules. It dynamically records user-selected symptoms, compares them with known fault symptom lists, calculates confidence scores, and recommends repair actions. The main Prolog strengths shown are logical inference, backtracking, list processing, dynamic facts, and rule-based decision making.

---

37. Quick Revision Checklist

Before your presentation, make sure you can explain:

- What `device/1` means
- What `symptom/3` means
- What `fault_info/7` stores
- What `fault_symptoms/3` stores
- Why `observed_symptom/1` is dynamic
- How `assertz/1` adds symptoms
- How `retractall/1` clears symptoms
- How recursion processes a symptom list
- How `member/2` checks list membership
- How `findall/3` collects matching symptoms
- How the score is calculated
- Why 50% is used as the threshold
- How backtracking finds multiple faults
- How repair decisions are made
- Why backup warning is important
- What limitations and future improvements exist

---

38. Best One-Minute Explanation

> My assignment is a Smart Device Repair Diagnosis Expert System using Prolog. It supports laptops and phones. I created facts for devices, symptoms, and fault information. I created diagnostic rules using `fault_symptoms/3`, where each fault has a list of symptoms. When a user selects symptoms, they are added dynamically using `assertz/1` as `observed_symptom/1` facts. The engine compares those symptoms with each fault rule using `member/2` and `findall/3`, counts the matches, calculates a confidence score, and returns likely faults above 50%. Prolog's backtracking allows it to find multiple possible faults. The system also uses cost, severity, and backup risk to recommend whether to repair, replace a part, or backup data first.